

Your Monte Carlo Simulation Toolkit: A Beginner's Guide

Written for Yamen Agha. Read it slowly, with the app and the Excel files open next to you.

This guide starts from zero. Your accounting courses taught you to measure what **has** happened. Risk management asks a different question: **what could happen next, and how bad could it get?** Monte Carlo simulation is the tool most finance teams use to answer it. By the end you should be able to:

1. Explain Monte Carlo simulation with dice and coins, and say why we need thousands of simulations.
2. Explain returns, volatility, correlation and geometric Brownian motion (GBM) in plain words.
3. Read a fan chart, and explain VaR, CVaR and maximum drawdown.
4. Explain a retirement "probability of success" and the 4% rule.
5. Explain what calls and puts are, what Black-Scholes does, and what each Greek means.
6. Build a small Monte Carlo simulation in Excel from an empty sheet.
7. Present the project to an employer, answer interview questions, and honestly say "I understand this and I improved it myself."

Every number in this guide was produced by the app's engine (`montecarlo.py`) with seed 42 unless stated otherwise. The portfolio figures use the offline price snapshot (5 years to 30 September 2026). Key results were also checked in LibreOffice.

Contents

1. Monte Carlo from zero: dice and coins
2. Randomness and the normal distribution
3. Returns: simple and log
4. Volatility and correlation
5. Geometric Brownian motion in plain words
6. Why so many simulations?
7. Percentiles and how to read a fan chart
8. VaR and CVaR
9. Drawdown
10. Retirement: probability of success and the 4% rule
11. Options: calls and puts
12. Black-Scholes intuition and the Greeks
13. Limitations: what Monte Carlo cannot do
14. How to use the app and the Excel files
15. Hands-on Excel exercise: build a Monte Carlo yourself
16. Presenting the project to an employer
17. Interview questions and sample answers

1. Monte Carlo from zero: dice and coins

Suppose someone asks: "If you roll two dice, what is the average total, and how often do you get a 7?"

You could work it out with probability: there are 36 equally likely outcomes, 6 of them total 7, so $P(7) = 6/36 = 16.67\%$, and the average total is 7.

Or you could **just roll the dice many times and count**. That is Monte Carlo simulation: when a problem is hard to solve with formulas, you simulate it with random numbers many times and look at the results. The computer rolled two dice for us:

Rolls	Average total	Share of 7s
10	7.50	10.0%
100	7.03	19.0%
1,000	6.996	17.5%
10,000	6.969	16.3%
100,000	7.010	16.9%

Two lessons are already here:

- **With few trials, the answer is noisy.** Ten rolls gave an average of 7.5 and only 10% sevens.
- **With many trials, the answer settles near the truth** (7 and 16.67%). This is the *law of large numbers*.

A coin works the same way: 10 tosses gave 40% heads, 100 gave 53%, 1,000 gave 50.0% and 10,000 gave 49.91%.

In finance, we don't know the formula for "what is my portfolio worth next year?" So we **simulate thousands of possible years** using random numbers that behave like market returns, then count: how many years ended in a loss? How bad was the worst 5%? That is all this app does, three times over.

The name comes from the Monte Carlo casino. Scientists working on the first computers in the 1940s used it as a code name for methods based on chance.

2. Randomness and the normal distribution

A die gives each number the same chance. Daily stock returns are different: small moves are common and big moves are rare. The usual starting model is the **normal distribution** (the

bell curve). It is described by two numbers:

- the **mean** (μ , "mu"): the centre
- the **standard deviation** (σ , "sigma"): how spread out it is

The useful rule of thumb for a normal distribution:

Range	Share of outcomes
mean $\pm 1\sigma$	about 68%
mean $\pm 2\sigma$	about 95%
mean $\pm 3\sigma$	about 99.7%

One more number you will see often: **1.645**. Only 5% of a normal distribution lies more than 1.645σ below the mean. For 1%, it's 2.326σ .

How a computer draws a normal random number: it draws a uniform number between 0 and 1 (like `RAND()` in Excel) and converts it with the inverse of the normal distribution (like `NORM.INV(RAND(), mean, sd)`). You will do exactly this in section 15.

3. Returns: simple and log

If a price goes from 100 to 110:

- the **simple return** is $110/100 - 1 = 10\%$
- the **log return** is $\ln(110/100) = 9.53\%$

Why bother with log returns? **They add up over time.** If you earn log returns of $r_1, r_2, \dots, r_{252}$ over a year, your total growth is $e^{(r_1 + r_2 + \dots + r_{252})}$. Simple returns don't add: +50% then -50% is not 0%. It is $1.5 \times 0.5 = 0.75$, a **25% loss**.

The app uses log returns for estimating and simulating, and converts back to money at the end. It annualises with **252 trading days** a year: mean $\times 252$, volatility $\times \sqrt{252}$.

4. Volatility and correlation

Volatility is the standard deviation of returns: how much they jump around. Over the 5 years to 30 September 2026:

	SPY (US stocks)	AGG (US bonds)
Daily volatility	1.08%	0.39%
Annual volatility ($\times \sqrt{252}$)	17.1%	6.1%
Average annual log return	12.67%	-0.69%

Yes, bonds lost money on average over this window. Interest rates rose sharply in 2022, and bond prices fall when rates rise. Keep this in mind for section 13: the simulation assumes the future looks like the window you chose.

Why $\times \sqrt{252}$ and not $\times 252$? Variances add over independent days, so variance $\times 252$, which means volatility $\times \sqrt{252}$. This **square-root-of-time rule** comes back in VaR.

Correlation measures whether two assets move together. It runs from -1 (always opposite) through 0 (unrelated) to $+1$ (always together). SPY and AGG had a correlation of **0.22**: slightly together, but far from 1 .

This is why diversification works. A 60/40 mix of SPY and AGG had a portfolio volatility of **11.1%**. If the correlation were 1 , it would be the weighted average, $0.6 \times 17.1\% + 0.4 \times 6.1\% = \mathbf{12.7\%}$. Low correlation removes risk for free.

To simulate correlated assets, the app uses the **Cholesky decomposition**. It turns independent random numbers into random numbers with exactly the covariance you measured. You don't need the algebra. Think of it as a recipe: "take a bit of the stock shock and mix it into the bond shock, in just the right amount." The tests check that the simulated correlations come back to the input within ± 0.01 .

5. Geometric Brownian motion in plain words

GBM is the standard model for a price that moves randomly. In words:

Each day, the price grows by a fixed average rate plus a random shock, and both are percentages of the current price.

Because the shocks are percentages, the price can never go below zero. A 2% fall from \$100 is \$2; from \$50, it is \$1.

There is one subtle point, and it caused the most important bug in the original code. If the average *simple* return is μ , the average *log* return is lower: $\mu - \sigma^2/2$. This is the "volatility drag". A +50% / -50% sequence averages 0% in simple terms but loses 25%.

- SPY's average log return was 12.67% a year, and its arithmetic drift (μ) was $12.67\% + 17.1\%^2/2 = \mathbf{14.14\%}$.
- The original code took the 12.67% (already a log mean) and subtracted $\sigma^2/2$ again. That understated SPY's growth by about 1.5 percentage points a year and made the risk look worse than it was.

The same drag is why the retirement module's "7% expected return" gives a **median year of 5.80%** when volatility is 15%: $\ln(1.07) - 0.15^2/2 = 5.64\%$ in log terms, and $e^{0.0564} - 1 = 5.80\%$.

6. Why so many simulations?

A simulation estimate has its own error, just like the dice. The rule is simple: **the error shrinks with the square root of the number of simulations**. Ten times more simulations makes the error about 3.16 times smaller ($\sqrt{10}$).

Example: pricing the textbook call option (section 12) by Monte Carlo:

Simulations	Price	Standard error	95% confidence interval
1,000	10.0204	0.3204	9.3925 to 10.6483
10,000	10.3425	0.1046	10.1376 to 10.5474
100,000	10.4567	0.0330	10.3921 to 10.5213

The exact answer is 10.4506, and every interval contains it. The standard error drops by about 3.1 each time.

Tail numbers such as VaR are noisier, because only 5% of the simulations sit in the tail. The 60/40 portfolio's 95% VaR was \$9,935 with 100 simulations, \$10,256 with 1,000, \$9,825 with 10,000 and \$10,250 with 50,000. So report VaR as "about \$10,000", not "\$9,825.13 exactly". Use **at least 10,000 simulations**, and always fix the **seed** so others can reproduce your number.

7. Percentiles and how to read a fan chart

A **percentile** says how many outcomes lie below a value. The 5th percentile is the value that 5% of simulations fall below. The 50th percentile is the **median**: half above, half below.

For \$100,000 in 60/40 SPY/AGG over one year (10,000 simulations):

Percentile	Final value
5th	\$90,175
25th	\$100,073
50th (median)	\$108,068
75th	\$116,973
95th	\$132,008

The mean was \$109,117, a bit above the median. This is normal: losses are limited to -100% but gains are not, so the distribution leans right.

A **fan chart** draws these percentiles for every day of the horizon:

- **The line in the middle** is the median path.

- **The inner (brighter) band** covers the 25th to 75th percentiles: half of all futures lie inside it.
- **The outer (fainter) band** covers the 5th to 95th percentiles: 90% of futures lie inside it.
- **The fan widens over time**, because uncertainty grows with the horizon (roughly with $\sqrt{\text{time}}$).

A fan chart does **not** show a single real path. Real paths wiggle across bands. And 10% of all paths go outside the outer band, half of them below it.

8. VaR and CVaR

Value at Risk (VaR) answers: *"How much could I lose, at a given confidence level, over a given horizon?"*

1-year 95% VaR = \$9,825 means: in 95% of the simulated years, the loss was smaller than \$9,825. In the worst 5%, it was larger.

It is just the 5th percentile turned into a loss: $\$100,000 - \$90,175 = \$9,825$ (9.83% of the starting value). Always say three things: **the amount (in \$ and %)**, **the confidence level** and **the horizon**. A "95% VaR" without a horizon means nothing. One-day VaR and one-year VaR are completely different numbers.

The weakness of VaR: it tells you where the bad 5% *begins*, not how bad it gets. **CVaR** (Conditional VaR, also called Expected Shortfall) fixes that:

1-year 95% CVaR = \$13,917 means: *in the worst 5% of years, the average loss was \$13,917 (13.92%).*

CVaR is always at least as big as VaR. Banking regulators (the Basel rules) moved from VaR to Expected Shortfall for this reason.

The 60/40 portfolio, \$100,000, 1 year, from the app:

Method	95% VaR	95% CVaR
Monte Carlo (correlated GBM)	\$9,825 (9.83%)	\$13,917 (13.92%)
Parametric (lognormal formula)	10.04%	14.04%
Historical (1,002 overlapping 1-year windows)	12.75%	14.75%
1-day historical VaR $\times \sqrt{252}$	17.03%	n/a

What to learn from this table:

- **GBM and the parametric formula agree**, because they make the same normal assumption. The formula is $VaR = 1 - e^{(H\mu - 1.645\sigma\sqrt{H})}$, with the daily mean and volatility scaled to $H = 252$ days.
- **History is worse**. Real 1-year windows include 2022, when stocks and bonds fell together. But the 1,002 windows overlap almost completely, so there are only about 5 truly independent years in them.
- **The $\sqrt{T}$ rule overstates the 1-year loss**, because it scales up only the bad side of one day and ignores a year of expected growth. It is fine for 1 to 10 days, which is what regulators use it for.

9. Drawdown

A **drawdown** is a fall from the highest value so far. The **maximum drawdown** of a path is its worst peak-to-trough fall. If a path goes $100 \rightarrow 120 \rightarrow 90 \rightarrow 130 \rightarrow 104$, the drops are $120 \rightarrow 90$ (25%) and $130 \rightarrow 104$ (20%), so the max drawdown is **25%**.

Why it matters: an investor who sees 25% disappear often panics and sells, even if the year ends well. A path can finish *up* and still have a painful drawdown along the way.

For the 60/40 portfolio over one year, the app computes the max drawdown on **each of the 10,000 paths**. The median was **9.1%**, the 95th percentile was **17.2%**, and the worst path fell **31.5%**.

10. Retirement: probability of success and the 4% rule

The retirement module simulates a whole life plan, month by month:

1. **Saving:** each month the balance earns a random return, then you add your contribution.
2. **Spending:** each month you take out 1/12 of the year's withdrawal first, then the rest earns a random return. The withdrawal rises with inflation once a year.

A plan **succeeds** if every withdrawal is paid in full until the end. The **probability of success** is the share of the 10,000 simulated lives in which that happened.

The sample plan: \$50,000 today, \$1,000 a month for 25 years, then \$60,000 a year (rising 2.5% a year) for 30 years. Returns 7% a year on average with 15% volatility.

Result	Value
Money you put in	$\$50,000 + 300 \times \$1,000 = \$350,000$
Nest egg if returns were exactly 7% every year	\$1,054,414
Median simulated nest egg	\$889,155 (lower, because of volatility drag)
5th–95th percentile of the nest egg	\$379,110 to \$2,351,361
First withdrawal as % of the median nest egg	6.75%
Withdrawal in the last year ($60,000 \times 1.025^{29}$)	\$122,784
Probability of success	34.3% (6,567 of 10,000 lives ran out)

So this plan fails two times in three. For the failed lives, the median time to run out was 38.7 years from today, about 13.7 years into retirement. What helps (each changed alone):

Change	Success
Withdraw \$50,000 instead of \$60,000	44.0%
Withdraw \$40,000	56.6%
Save \$1,500 a month instead of \$1,000	52.1%
Save \$2,000 a month	65.9%
Work 30 years instead of 25	53.2%

The 4% rule. In 1994, William Bengen looked at US history and found that a retiree with a stock/bond mix who withdrew 4% of the portfolio in the first year, then raised the amount with inflation, never ran out of money over any 30-year period in his data. In this app's model (7% return, 15% volatility, 2.5% inflation), \$1,000,000 with a \$40,000 first withdrawal succeeds in **70.8%** of 10,000 lives. With 10% volatility, it succeeds in 88.1%. The rule of thumb depends a lot on what you assume.

The **safe-withdrawal-rate sweep** runs the same test for rates from 2% to 8% on the median nest egg (\$889,155), always with the same random numbers so the rates are compared fairly. In 30 years: 3% gives 87.4%, 4% gives 69.9% and 5% gives 50.4%. The highest rate with at least 90% success is 2.5%.

11. Options: calls and puts

An **option** is a contract that gives the right, but not the obligation, to trade an asset at a fixed price (the **strike**, K) on a fixed date (the **expiry**, T).

- A **call** is the right to **buy** at K. You profit if the price goes up. Payoff at expiry: $\max(S_T - K, 0)$. If the stock ends at 120 and $K = 100$, the call pays 20. If it ends at 90, you simply don't use it, so it pays 0.
- A **put** is the right to **sell** at K. You profit if the price goes down. Payoff: $\max(K - S_T, 0)$. If the stock ends at 80, a $K = 100$ put pays 20. Puts work like insurance on a

portfolio.

The app prices **European** options, which can only be used at expiry.

The buyer pays a **premium** for this one-sided bet. The question is: what is the fair premium today? Monte Carlo answers it directly:

1. Simulate thousands of possible prices at expiry with GBM. In option pricing we use the **risk-free rate** r as the drift, not the stock's own expected return. This is the "risk-neutral" trick that makes the price independent of anyone's opinion.
2. Compute the payoff in each simulation.
3. Average the payoffs and **discount** them to today: multiply by $e^{(-rT)}$.

Antithetic variates is a free accuracy boost. For every random draw Z , the app also uses $-Z$. A lucky path and its mirror image cancel out some of the noise. With 10,000 simulations, the antithetic price was 10.3425 (SE 0.1046), and a plain estimate with independent draws was 10.4290 (SE 0.1443). That is a variance reduction of 1.91×: the antithetic method needs about half as many payoff calculations for the same accuracy.

One detail matters: the pair $(Z, -Z)$ is not two independent samples. So the standard error must be computed on the **5,000 pair averages**, not on 10,000 payoffs. The app does this, and a test checks it.

12. Black-Scholes intuition and the Greeks

For European options under GBM, Fischer Black, Myron Scholes and Robert Merton found an exact formula (1973). The Monte Carlo answer converges to it, which is how the app checks itself.

The textbook case: $S = 100$, $K = 100$, $r = 5\%$, $\sigma = 20\%$, $T = 1$ year, no dividends.

- $d1 = [\ln(S/K) + (r + \sigma^2/2)T] / (\sigma\sqrt{T}) = (0 + 0.07) / 0.2 = \mathbf{0.35}$
- $d2 = d1 - \sigma\sqrt{T} = \mathbf{0.15}$
- $N(d1) = 0.6368$ and $N(d2) = 0.5596$, where N is the normal cumulative distribution (`NORM.S.DIST(x, TRUE)` in Excel)
- **Call** = $S \cdot N(d1) - K \cdot e^{(-rT)} \cdot N(d2) = \mathbf{10.4506}$
- **Put** = $K \cdot e^{(-rT)} \cdot N(-d2) - S \cdot N(-d1) = \mathbf{5.5735}$

Intuition. The call is "what you expect to receive (the stock, weighted by $N(d1)$) minus what you expect to pay (the strike, discounted and weighted by $N(d2)$)". $N(d2) = 56\%$ is the risk-neutral probability that the call ends in the money.

Put-call parity links the two: $\text{Call} - \text{Put} = S - K \cdot e^{(-rT)}$. Here $10.4506 - 5.5735 = 4.8771 = 100 - 95.1229$. The Excel sheet checks this in a cell.

The Greeks measure how the price reacts when one input changes:

Greek	Meaning	Textbook call	Textbook put
Delta	Price change per \$1 move in the stock	0.6368	-0.3632
Gamma	Change in delta per \$1 move in the stock	0.0188	0.0188
Vega	Price change per 1 percentage point of volatility	0.3752	0.3752
Theta	Price change per calendar day that passes	-0.0176	-0.0045
Rho	Price change per 1 percentage point of interest rate	0.5323	-0.4189

Check vega yourself: at 25% volatility the call costs 12.3360, which is 1.8854 more. That is close to 5 points $\times 0.3752 = 1.876$. (Vega itself changes a little as volatility moves, so it isn't exact.)

The app also estimates delta and vega from the simulation (the "pathwise" method): 0.6381 and 36.85 per 1.00 of volatility, against the exact 0.6368 and 37.52.

An Excel trap I fixed. The normal density is $e^{-(d1^2/2)}/\sqrt{2\pi}$. In Excel, `=EXP(-B14^2/2)` is wrong: Excel applies the minus sign **before** the power, so it computes $(-d1)^2$, and the result is $e^{+(d1^2/2)}$. The workbook uses `NORM.S.DIST(d1, FALSE)` instead.

13. Limitations: what Monte Carlo cannot do

1. **Garbage in, garbage out.** The simulation is only as good as the mean, volatility and correlation you feed it. Over 5 years, AGG's average return was negative. Over 3 years, the 60/40 one-year 95% VaR is only \$1,704, and over 10 years it is \$8,661. The same portfolio, three different answers. Always state the data window.
2. **Fat tails.** Real markets have more extreme days than the normal distribution predicts. GBM underestimates crashes like 1987, 2008 or March 2020. The historical bootstrap keeps real fat tails, but only the ones that happened in your window.
3. **The past is not the future.** Correlations change. In a crisis, many assets fall together. In 2022, stocks and bonds both fell.
4. **Constant parameters.** GBM keeps volatility constant. Real volatility comes in clusters (calm years, wild months). The block bootstrap keeps some of that.
5. **Precision is not accuracy.** 50,000 simulations make the answer *stable*, not *true*. A precise answer to the wrong model is still wrong.
6. **Black-Scholes assumptions:** constant volatility and interest rate, no transaction costs, continuous trading, lognormal prices. Real option markets show a "volatility smile", which Black-Scholes cannot produce.
7. **Not modelled here:** fees, taxes, changing spending, pensions, sequence-of-return strategies, and American options.

14. How to use the app and the Excel files

Starting it

```
cd portfolio/finance/monte-carlo-simulator
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py
```

Step by step

1. **Sidebar.** Choose the module, the number of simulations and the seed. Keep "Fixed random seed" ticked to get the same numbers each time.
2. **Portfolio risk.** Choose a preset or type tickers and weights (any scale, e.g. "60, 40"). Choose the data source: Yahoo Finance (live) or the offline snapshot (to 30 September 2026). Then the history length, starting value, horizon in trading days ($252 = 1$ year), method (GBM, i.i.d. bootstrap or block bootstrap), expected return (historical or zero) and rebalancing. The caption under the inputs shows the exact data dates. The tabs show the fan chart, the distribution with VaR lines, the comparison of VaR methods, drawdowns, and the estimated inputs.
3. **Retirement plan.** Enter savings, contribution, years, withdrawal, inflation, return and volatility. Tick "Withdrawal is in today's money" if the \$60,000 is meant in today's prices. (Then it becomes \$111,237 in year 26, and success falls to 9.6%.) The tabs show the fan chart (nominal or today's money), the safe-withdrawal-rate sweep and when failed plans run out.
4. **Option pricing.** Enter S , K , T , r , q , σ and the type. The cards show Black-Scholes, the Monte Carlo price with its 95% CI, and the variance reduction. The tabs show the convergence, the payoff histogram and the Greeks.

The Excel workbooks

Open `examples/Option_BS_vs_MC.xlsx` first.

- **Black-Scholes sheet:** blue cells on yellow (B5:B11) are the inputs. $d1$, $d2$, $N(d1)$, $N(d2)$, the call, the put, the parity check and all Greeks are **formulas**. Change σ from 20% to 25% and watch the call become 12.3360.
- **Monte Carlo sheet:** each row is one random draw z (exported, so it doesn't change). S_T , the payoffs and the pair average are formulas, and the price, SE and CI at the top are `AVERAGE` and `STDEV.S` formulas over them.

`examples/Portfolio_60_40_SPY_AGG.xlsx` : the Summary sheet computes VaR as `=B5 - PERCENTILE.INC(final values, 5%)` and CVaR with `AVERAGEIF` over **all 10,000** simulated final values on the Simulations sheet. The History sheet has the real prices, `LN` returns and `CORREL`. An audit block shows the Python value next to each Excel result.

examples/Retirement_Plan.xlsx : the Summary has the success probability as COUNTIF over the 10,000 lives, and an FV check of the no-volatility nest egg (\$1,054,413.51).

15. Hands-on Excel exercise: build a Monte Carlo yourself (45-60 minutes)

Goal: simulate one year of a \$10,000 investment with a 7% expected return and 15% volatility, then measure its VaR. Do it in a blank workbook.

Part A: one simulation per row

1. Inputs (make them blue):

Cell	Label	Value
B1	Start value	10000
B2	Expected return	7%
B3	Volatility	15%
B4	Log drift	$=\text{LN}(1+B2) - B3^2/2 \rightarrow 0.05641$

B4 is the GBM drift correction from section 5. 2. In A7 type "Sim", in B7 "Final value". In A8:A1007 put the numbers 1 to 1000. 3. In B8: $=\$B\$1 * \text{EXP}(\text{NORM.INV}(\text{RAND}(), \$B\$4, \$B\$3))$. Copy it down to B1007. Each row is one possible year. 4. Results:

Cell	Formula	Theory
E1 Mean	$=\text{AVERAGE}(B8:B1007)$	10,700
E2 Median	$=\text{MEDIAN}(B8:B1007)$	10,580.30
E3 5th percentile	$=\text{PERCENTILE.INC}(B8:B1007, 0.05)$	8,266.95
E4 95% VaR	$=B1 - E3$	1,733.05
E5 95% CVaR	$=B1 - \text{AVERAGEIF}(B8:B1007, "<="&E3)$	2,223.56
E6 P(loss)	$=\text{COUNTIF}(B8:B1007, "<"&B1) / 1000$	35.3%

- Press **F9** a few times. Every number changes, because $\text{RAND}()$ draws new numbers on every recalculation. That is the simulation noise from section 6. With one set of 1,000 draws, the engine got a mean of 10,651, a median of 10,590, a 5th percentile of 8,217, a CVaR of 2,263 and P(loss) of 34.5%. Yours will be different but close.
- Why the "theory" column? Here the answer is known exactly: the 5th percentile is $10,000 \times e^{(0.05641 - 1.645 \times 0.15)} = 8,266.95$. This is the same parametric formula as in section 8. Comparing a simulation with a known answer is how you test any model.

The app's workbooks export the random draws as **values** instead of using $\text{RAND}()$, so the results stay reproducible and match Python.

Part B: a data table instead of 1,000 formulas

Excel's **data table** can repeat one simulation many times:

1. On a new sheet, put the inputs in B1:B4 as before and one simulation in B6:
`=B1*EXP(NORM.INV(RAND(),B4,B3))`.
2. In D2:D1001 type 1 to 1000. In E1 type `=B6`.
3. Select D1:E1001, then **Data → What-If Analysis → Data Table**. Leave "Row input cell" empty, put any **empty** cell (e.g. G1) as the "Column input cell", and click OK.
4. Excel fills E2:E1001 with 1,000 independent results of B6. Compute the same statistics as in Part A on E2:E1001.

Stretch goals

- Add a second asset with a correlation: $z_2 = \rho \cdot z_1 + \sqrt{(1-\rho^2)} \cdot z_3$, where z_1 and z_3 are independent `NORM.S.INV(RAND())`. That is Cholesky for two assets.
- Price the textbook call: $S_T = 100 \cdot \exp((0.05 - 0.2^2/2) \cdot 1 + 0.2 \cdot \text{NORM.S.INV(RAND())})$, payoff `=MAX(S_T-100,0)`, price `=EXP(-0.05)*AVERAGE(payoffs)`. Compare it with 10.4506.

16. Presenting the project to an employer

60-second pitch

"I built a Monte Carlo simulation toolkit in Python and Streamlit with three modules. The first downloads real prices and simulates thousands of correlated portfolio paths, then reports VaR, CVaR and drawdowns and compares them with parametric and historical VaR. The second tests whether a retirement plan survives, and finds the safe withdrawal rate. The third prices options by simulation and checks them against Black-Scholes. The part I'm proudest of is the model review. I reviewed the first draft and found real errors in it: the drift was corrected twice, missing tickers were replaced with made-up random data, and two 'different' estimators were secretly the same numbers. I fixed them with tests, and the Excel exports rebuild every statistic as live formulas that match Python in LibreOffice, across 47,000 values. It has 64 automated tests."

5-minute demo script

1. **(45 s)** Portfolio risk, 60/40 SPY/AGG, \$100,000, 1 year. "In 95% of simulated years we lose less than \$9,825. In the worst 5%, we lose \$13,917 on average." Point to the data date in the caption.
2. **(45 s)** Open the fan chart: "half of all futures are in the inner band." Then the VaR comparison tab: "history is worse than the normal model, because 2022 hit stocks and bonds at the same time."
3. **(45 s)** Switch to the bootstrap method and show how VaR changes. Point to the drawdown tab.

4. **(60 s)** Retirement: "\$50,000 plus \$1,000 a month, then \$60,000 a year: only 34.3% of lives make it." Show the safe-withdrawal sweep and the 4% point.
5. **(45 s)** Options: Black-Scholes 10.4506. "The Monte Carlo interval 10.14 to 10.55 contains it. The convergence chart shows the band shrinking with $\sqrt{n}$."
6. **(40 s)** Open the Excel file and change σ on the Black-Scholes sheet. Close with the bug list in the README and the tests.

CV bullet points

- Built a Python/Streamlit Monte Carlo toolkit for **portfolio VaR/CVaR** (correlated GBM via Cholesky and historical bootstrap on real Yahoo Finance data), **retirement success probability** and **option pricing** (antithetic variates vs Black-Scholes, Greeks).
- Reviewed the first version like a model validator and **fixed 10+ defects**, including a double Itô drift correction, synthetic data silently replacing missing tickers, and a misstated confidence interval; added 64 pytest tests with regressions for every fix.
- Produced Excel exports whose statistics (`PERCENTILE.INC` , `AVERAGEIF` , `STDEV.S`) and Black-Scholes sheet are **live formulas**, verified in LibreOffice against Python: **47,233 values, 0 mismatches**.

LinkedIn post draft

🤖 How much could a "safe" 60/40 portfolio lose in a year?

I simulated 10,000 possible years for \$100,000 in 60% S&P 500 (SPY) / 40% US bonds (AGG), using 5 years of real prices to 30 Sep 2026: • in 95% of years, the loss is under \$9,825 (that's the 95% VaR) • in the worst 5%, the average loss is \$13,917 (CVaR) • real 1-year windows from history are worse, because in 2022 stocks and bonds fell together

I built this Monte Carlo toolkit in Python + Streamlit, with a retirement planner (is the 4% rule safe?) and an option pricer checked against Black-Scholes.

My favourite lesson: reviewing the first draft found real model errors, such as volatility drag subtracted twice and made-up prices for unknown tickers. Every Excel export is now built from live formulas and matches Python.

Code and a beginner's guide: github.com/Yasskik/portfolio

RiskManagement #MonteCarlo

#Finance #Python #Excel

17. Interview questions and sample answers

- 1. What is Monte Carlo simulation?** Solving a problem by simulating it many times with random numbers and looking at the distribution of results. In finance, it's used when the outcome depends on random returns and no simple formula exists, for example portfolio risk, retirement plans or complex derivatives.
- 2. How many simulations do you need?** Enough that the answer is stable. The error falls with $\sqrt{n}$: 10× more simulations makes it about 3.2× smaller. I use at least 10,000, fix the seed, and report a confidence interval. Tail numbers like 99% VaR need more simulations than averages.
- 3. What is VaR, and what is wrong with it?** VaR at 95% over one year is the loss that is exceeded in only 5% of years. It always needs a confidence level and a horizon. Its weakness: it says nothing about how bad the tail is, and it isn't always sub-additive, so it can penalise diversification. CVaR, the average loss beyond VaR, fixes both.
- 4. Why use log returns?** They add over time, so a year's log return is the sum of the daily ones, and a price modelled with them can never go negative. The catch is that the mean log return is lower than the mean simple return by about $\sigma^2/2$.
- 5. What is the Itô (volatility drag) correction?** In GBM, the expected log return is $\mu - \sigma^2/2$, where μ is the arithmetic drift. If you estimate the mean of log returns, the correction is already in it. Subtracting $\sigma^2/2$ again was a bug in the first version.
- 6. Why the Cholesky decomposition?** To turn independent random numbers into correlated ones with exactly the measured covariance: if $\Sigma = LL^T$, then $L \cdot z$ has covariance Σ . I tested that the simulated correlations recover the input.
- 7. GBM or historical bootstrap?** GBM is smooth and normal, so it is easy to explain but has thin tails. Bootstrapping real days keeps fat tails and cross-asset relationships, but it can only replay what happened in the window. Resampling single days breaks volatility clustering, so a block bootstrap of about a month keeps some of it.
- 8. Why can historical, parametric and Monte Carlo VaR disagree?** They make different assumptions. Parametric and GBM assume normal log returns. Historical uses the real path, including events like 2022. With a 1-year horizon and 5 years of data, the overlapping windows contain only about 5 independent years. The $\sqrt{T}$ scaling of a 1-day VaR ignores the drift and overstates a 1-year loss.
- 9. What is the 4% rule and what are its limits?** Withdraw 4% of the portfolio in the first year of retirement, then increase the amount with inflation. Bengen found it survived every 30-year period in US history. In a model with 7% return and 15% volatility, it only succeeds about 71% of the time. It depends on returns, volatility, fees and the retirement length.
- 10. Explain the Black-Scholes formula without maths.** The call's value is the expected stock you receive minus the expected discounted strike you pay, both weighted by risk-

neutral probabilities. $N(d_2)$ is the probability of finishing in the money. For $S = K = 100$, $r = 5\%$, $\sigma = 20\%$, $T = 1$, the call is 10.4506 and the put 5.5735.

11. What are delta and vega? Delta is how much the option price moves for a \$1 move in the stock (0.64 for the textbook call); traders use it to hedge. Vega is the change per volatility point (0.375). Option prices rise with volatility, because the payoff is one-sided.

12. What are antithetic variates, and how do you compute their standard error? For every draw Z , also use $-Z$. The errors partly cancel, so the variance falls ($1.9\times$ here). The pair is not independent, so the standard error must be computed on the pair averages, not on all the payoffs. Otherwise the confidence interval is wrong.

Bonus: what bugs did you find? - The drift was corrected for volatility twice. - Missing tickers were replaced with synthetic random prices. - The "standard" and "antithetic" option estimates used the same random numbers, so they were identical. - "Historical VaR" was really a $\sqrt{T}$ -scaled 1-day number. - Greeks were zero at zero volatility, and the retirement return was 7.29% instead of 7%.

How to show you really know the project

- Do the Excel exercise in section 15 from an empty sheet, without help.
- Calculate d_1 , d_2 and the Black-Scholes call for the textbook case with a calculator.
- Explain the difference between the four VaR numbers in section 8.
- Know the conventions table in the README: log returns, 252 days, drift, Cholesky, rebalancing, VaR sign and horizon, retirement timing and success definition.

Improvements you can make yourself (pick 2-3)

1. **Student-t shocks:** replace the normal shocks with a t-distribution (4-5 degrees of freedom, scaled to the same volatility) and compare the 99% VaR. You'll see fat tails in action.
2. **Stress scenarios:** add a button that applies the 2008 or 2022 returns to today's portfolio.
3. **Kupiec backtest:** compute a 1-day 95% VaR on a rolling window and count how often real losses exceeded it. About 5% is right; many more means the model is too optimistic.
4. **Contributions in retirement planning that rise with salary,** or a pension that starts at a given age.
5. **Guardrails:** cut the withdrawal by 10% after a bad year and see how much the success probability improves.
6. **Asian or barrier options,** which have no simple formula, so Monte Carlo is the natural method. Use the European case as your test.
7. **Control variates:** use the stock price itself (whose expected value is known) to reduce the variance of the option price further, and compare with antithetic variates.

For each one, write the formula on paper first, add a test with a number you know (from Excel or a textbook), and only then write code.

18. Your study plan for the next 7 days

Day	Goal (about 1-1.5 hours)	Check yourself
1	Sections 1-3. Roll dice in Excel with <code>RANDBETWEEN(1,6)</code> 1,000 times.	Share of 7s near 16.67%; log return of 100 → 110 is 9.53%
2	Sections 4-6. Use the app's input tab; compute the 60/40 volatility by hand.	SPY 17.1%, AGG 6.1%, correlation 0.22, portfolio 11.1%
3	Sections 7-9. Run the portfolio module with GBM and bootstrap; explain each VaR.	95% VaR \$9,825, CVaR \$13,917, median drawdown 9.1%
4	Section 15, Parts A and B, from an empty sheet.	5th percentile near 8,267; P(loss) near 35%
5	Section 10. Run the retirement module; try the changes in the table.	34.3% success; \$50,000 withdrawal gives 44.0%
6	Sections 11-12. Calculate d1, d2 and the call by hand, then in the example workbook.	0.35, 0.15, 10.4506; put 5.5735; delta 0.6368
7	Sections 16-17. Rehearse the pitch and the demo out loud, and answer the 12 questions without notes. Start one improvement from section 17.	A friend can follow your demo in 5 minutes

Good luck. Once you can explain why 10,000 simulated years say "lose less than \$9,825 in 95% of them", and why history says something worse, you understand risk better than most people who quote VaR numbers.